

COMP 3311

DATABASE MANAGEMENT

SYSTEMS

LECTURE 4

RELATIONAL DATABASE DESIGN:

E-R SCHEMA TO RELATION SCHEMAS REDUCTION

RELATIONAL DATABASE DESIGN: OUTLINE

Relational Data Model

E-R Schema to Relation Schemas Reduction

Functional Dependencies

Normalization

RELATIONAL DATA MODEL

The relational data model represents the data for an application as a collection of tables¹.

Relational Data Model		Representation	Notation
Relation	↔	table	$R(A_1, A_2, \dots, A_n)$
Attribute	↔	column	A_i
Domain	↔	type and range of values	$\text{dom}(A_i)$
Tuple / Record	↔	row	
Attribute value	↔	value in a table cell	

Examples of attribute domains:

Attribute	Domain
age	[0-100]
name	50 alphabetic characters
salary	non-negative integer

1. A table is not a relation; it is just a convenient way for a user to view the data in a relational database.

RELATIONAL DATA MODEL: SCHEMAS & INSTANCES

- A set of **relation schemas** define a **relational database** and the data that can be stored in it.

Project(projectNo, name, budget) ← **relation schema**

Employee(empld, name, address, hkid, projectNo) ← **relation schema**

- A **table** is a convenient way to represent the **relation schemas** and their **instances** (i.e., their tuples).

Project

projectNo	name	budget
1	E-commerce	200,000
2	Stock control	100,000
3	Web store	500,000

Employee

empld	name	address	hkid	projectNo
1	Holmes D.	86 Queen	A450361	3
5	Chan B.	21 Minto	C461378	2
35	Hui J.	16 Peak	F562916	1
8	Bell G.	53 Water	A417394	2
15	Wing R.	58 Aster	C538294	3

RELATIONAL DATA MODEL: FORMAL DEFINITIONS

- A tuple t is an **ordered sequence** of n values $(v_1, v_2, \dots, v_n)$ such that $v_i \in \text{dom}(A_i)$ or is **null**.
- The **Cartesian product** over $\text{dom}(A_1), \text{dom}(A_2), \dots, \text{dom}(A_n)$ is the set of all possible tuples $\{t_1, t_2, \dots, t_m\}$ with first element $\in \text{dom}(A_1)$, second element $\in \text{dom}(A_2)$, ... and last element $\in \text{dom}(A_n)$.

Example: Let $\text{dom}(\text{name}) = \{\text{Lee}, \text{Cheung}\}$ and $\text{dom}(\text{grade}) = \{\text{A}, \text{B}, \text{C}\}$.

Then the Cartesian product of the two domains is

$$\text{dom}(\text{name}) \times \text{dom}(\text{grade}) = \{ \langle \text{Lee}, \text{A} \rangle, \langle \text{Lee}, \text{B} \rangle, \langle \text{Lee}, \text{C} \rangle, \\ \langle \text{Cheung}, \text{A} \rangle, \langle \text{Cheung}, \text{B} \rangle, \langle \text{Cheung}, \text{C} \rangle \}$$

☞ The Cartesian product **is not** commutative since **the domain order matters**.

- A relation instance (or simply relation) r over relation schema R is $r(R) \subseteq \text{dom}(A_1) \times \text{dom}(A_2) \times \dots \times \text{dom}(A_n)$.

☞ A relation instance is **any subset** of the Cartesian product of its domains.

Example: $r(R(\text{name}, \text{grade})) = \{ \langle \text{Lee}, \text{A} \rangle, \langle \text{Cheung}, \text{C} \rangle \} \subseteq \text{dom}(\text{name}) \times \text{dom}(\text{grade})$

RELATIONAL DATA MODEL: PROPERTIES OF RELATIONS

- Tuples in a relation are *not ordered*, even though they are represented in a tabular form.

👉 Recall that a relation is a set of tuples.

- All attribute values are *atomic*.

👉 Multivalued and composite attribute values are **not allowed** in a relation, although they are permitted in the E-R data model.

- The degree of a relation is the **number of attributes**.

- The cardinality of a relation is the **number of tuples**.

👉 This is not the same as the cardinality constraint in the E-R data model.

RELATIONAL DATA MODEL: KEYS

Superkey A **superkey**, S , of relation $R = \{A_1, A_2, \dots, A_n\}$ is a set of attributes $S \subseteq R$ such that for any two tuples t_1 and $t_2 \in r(R)$, $t_1[S] \neq t_2[S]$.

✎ A superkey is **any** set of attributes that can uniquely identify a tuple in $r(R)$; *it does not have to be minimal*.

Employee(empId, name, address, hkid, projectNo)
where empId and hkid are unique.

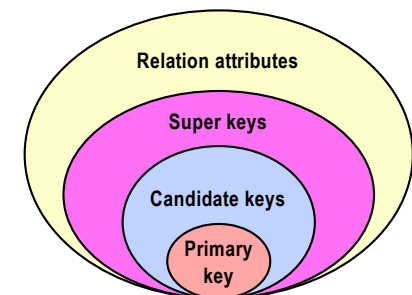
Possible superkeys: empId
hkid
{empId, name}
{hkid, name, address}
plus, many others

Relational Database Design Note

- In the **relational data model**, every relation is **required** to have a candidate key.
- In a **relational DBMS**, a relation is **not required** to have a candidate key.

Candidate key A superkey that is **minimal**.
(A relation may have **several candidate keys**.)

Primary key One of the candidate keys.
(A relation can have **at most one primary key** —selected by the database designer.)



RELATIONAL DATA MODEL: CONSTRAINTS

Entity integrity constraint

If X is a primary key for R , then X cannot contain null values.

Referential integrity (foreign key) constraint



Given two relations S and T , relation T may reference relation S via a set of attributes fk_S that forms the primary key k_S of S .

The attributes fk_S in T are called a foreign key.

The value of the foreign key fk_S in a tuple of T must either be equal to the value of the primary key k_S of a tuple in S or be entirely null.

✎ A foreign key represents a 1:1 or 1:N relationship.

RELATIONAL DATA MODEL: CONSTRAINTS (CONTD)

Referential integrity (foreign key) constraint

The attribute **projectNo** in Employee is a **foreign key** since it references the primary key **projectNo** of Project (to indicate the project associated with an employee).

Project(projectNo, name, budget)

Employee(empld, name, address, hkid, **projectNo**)

Project

<u>projectNo</u>	name	budget
1	E-commerce	200,000
2	Stock control	100,000
3	Web store	500,000

Values of **projectNo** in
Employee must be equal to
projectNo in Project or be null.

Employee

<u>empld</u>	name	address	hkid	projectNo
1	Holmes D.	86 Queen	A450361	3
5	Chan B.	21 Minto	C461378	2
35	Hui J.	16 Peak	F562916	1
8	Bell G.	53 Water	A417394	2
15	Wing R.	58 Aster	C538294	3

➡ This represents a 1:N relationship.

E-R SCHEMA TO RELATION SCHEMAS REDUCTION: OVERVIEW

We need to reduce:

- generalizations / specializations** ⇒ inheritance, coverage
- attributes** ⇒ composite, multivalued
- entities** ⇒ strong, weak
- relationships** ⇒ degree (e.g., unary, binary)
⇒ constraints (cardinality, participation, inclusion)

**Cardinality/participation constraints in an E-R schema
reduce to
referential integrity constraints in a relation schema.**

E-R SCHEMA TO RELATION SCHEMAS REDUCTION: REFERENTIAL INTEGRITY ACTIONS



If relation **T** contains an attribute fk_S that is the primary key k_S of relation **S**, which can be specified as the foreign key constraint

foreign key (fk_S) references **S**(k_S)

then the value of fk_S in a tuple of **T** must either be equal to the value of the primary key k_S of a tuple in **S** or be entirely null.

To enforce this constraint, relation schemas require the following actions.

If in an E-R schema the referencing entity has total participation

on delete cascade - Delete all **T** tuples with foreign key values that match the primary key value of the deleted **S** tuple.

If in an E-R schema the referencing entity has partial participation

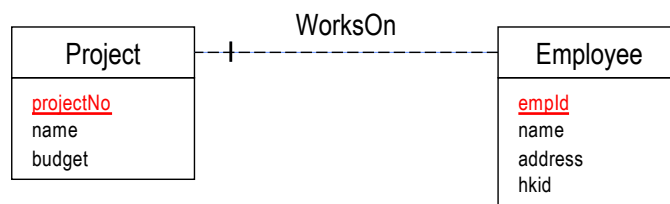
on delete set null - Set to null the foreign key value of all **T** tuples whose foreign key value matches the primary key value of the deleted **S** tuple.

E-R SCHEMA TO RELATION SCHEMAS REDUCTION: REFERENTIAL INTEGRITY ACTIONS

If in an E-R schema the referencing entity has total participation

Suppose project 3 is deleted.

What action applies to the Employee relation — **cascade** or **set null**?
⇒ on delete cascade



Project(projectNo, name, budget)

Employee(empld, ..., projectNo)

Project

<u>projectNo</u>	name	budget
1	E-commerce	200,000
2	Stock control	100,000
3	Web store	500,000

Employee

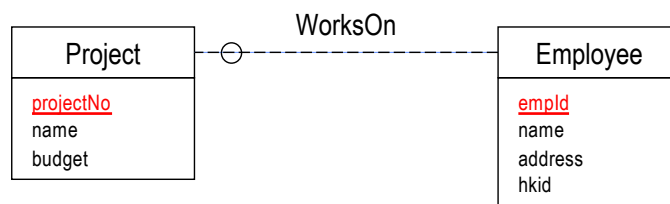
<u>empld</u>	name	address	hkid	<u>projectNo</u>
1	Holmes D.	86 Queen	A450361	3
5	Chan B.	21 Minto	C461378	2
35	Hui J.	16 Peak	F562916	1
8	Bell G.	53 Water	A417394	2
15	Wing R.	58 Aster	C538294	3

E-R SCHEMA TO RELATION SCHEMAS REDUCTION: REFERENTIAL INTEGRITY ACTIONS

If in an E-R schema the referencing entity has **partial participation**

Suppose project 3 is deleted.

What actions apply to the Employee relation — **cascade** or **set null**?
⇒ on delete set null



Project(projectNo, name, budget)

Employee(empld, ..., projectNo)

Project

<u>projectNo</u>	name	budget
1	E-commerce	200,000
2	Stock control	100,000
3	Web store	500,000

Employee

<u>empld</u>	name	address	hkid	<u>projectNo</u>
1	Holmes D.	86 Queen	A450361	3
5	Chan B.	21 Minto	C461378	2
35	Hui J.	16 Peak	F562916	1
8	Bell G.	53 Water	A417394	2
15	Wing R.	58 Aster	C538294	3

E-R SCHEMA TO RELATION SCHEMAS REDUCTION: SURROGATE KEY

- A **surrogate key** is a *new attribute* introduced into an entity to be the **primary key** for the entity.

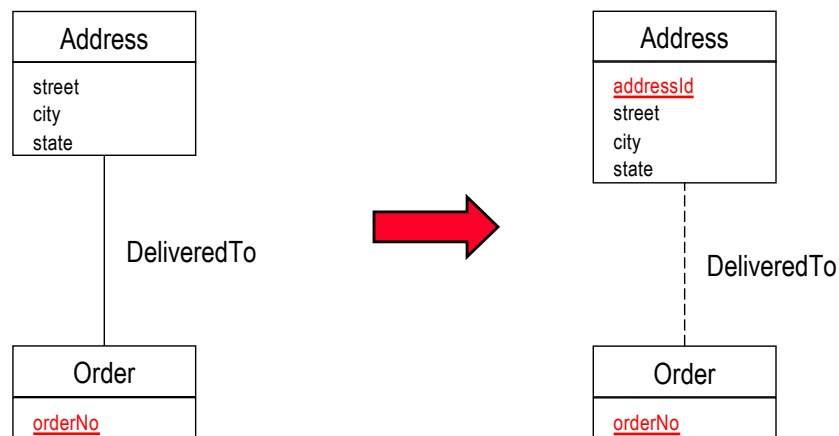
✎ It normally has **no meaning** in the application domain.

- Surrogate keys are used to **make weak entities strong** or to **replace a strong entity's key**, if it consists of many attributes, with a **single attribute**.

✎ To facilitate **querying and schema reduction**.

✎ A **surrogate key** is usually **invisible** to the user.

The values of a surrogate key are usually sequentially assigned integers.



Database Design Note
Surrogate keys are **never** introduced into an E-R schema.

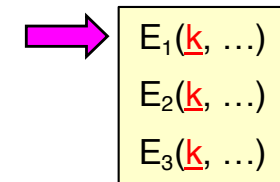
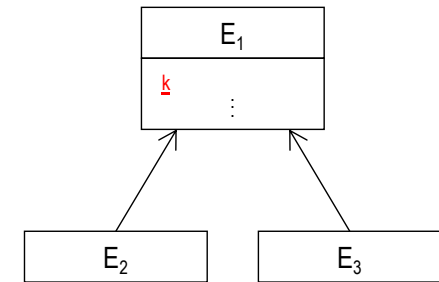
REDUCING GENERALIZATIONS/SPECIALIZATIONS

Option 1: Reduce all entities to relation schemas.

- Create a relation schema **for each entity** (supertype and subtype).
- For each relation schema created for a subtype entity:

Add: 1. the **primary key**, **k**, of the supertype entity as a foreign key **fk**.

- The foreign key **fk** becomes the primary key.
- 2. a **foreign key constraint**:
foreign key (fk) references supertype-relation(**k**).
- 3. a **referential integrity action**: on delete cascade.

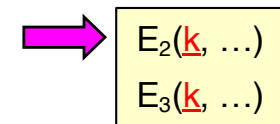


Option 2: Reduce only subtype entities to relation schemas.

- Create a relation schema **for each subtype entity**.
- For each relation schema created for a subtype entity:

Add: 1. all the attributes of the supertype entity.

- The primary key is the primary key for the supertype entity.

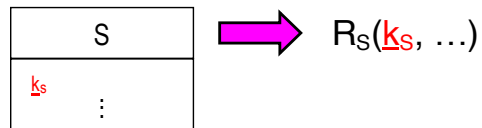


👉 **Option 2 should be used only for total, disjoint generalizations/specializations!**

REDUCE STRONG ENTITIES

For a strong entity **S**:

- Create a relation schema R_S with all the attributes of entity **S**, excluding multivalued and composite attributes.
- The primary key for relation R_S is the primary key for entity **S**.

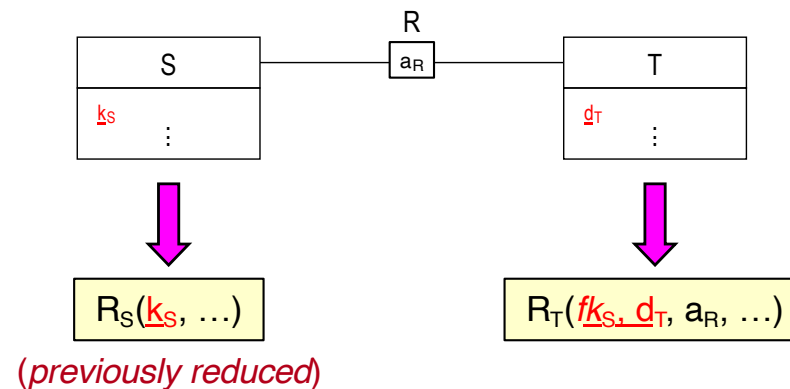


REDUCE WEAK ENTITIES

For a weak entity **T** that depends on a strong entity **S**:

- Create a relation schema R_T with attributes of the weak entity **T**.
- Include attributes a_R of relationship **R** in relation R_T .
- Include as foreign key attributes fk_S in relation R_T , the primary key attributes k_S of the strong entity **S**.
- The primary key for relation R_T is the **union of the foreign key attributes fk_S and the discriminator d_T** , if any, for the weak entity **T**.

Add: 1. a **foreign key constraint**: foreign key (fk_S) references **S**(k_S).
 2. a **referential integrity action**: on delete cascade.



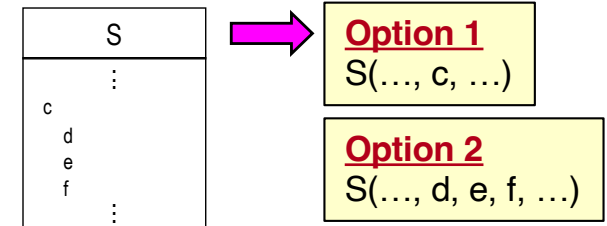
If a weak entity is dependent on several strong entities, include the key for each strong entity as a foreign key, as well as any relationship attributes, in the weak entity.

REDUCING COMPOSITE/MULTIVALUED ATTRIBUTES

For a composite attribute **C** in a strong entity **S**:

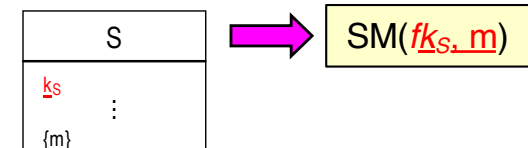
Option 1: Create a single attribute by “concatenating” the components of the composite attribute.

Option 2: Create a separate attribute for each component of the composite attribute.



For a multivalued attribute **M** in a strong entity **S**:

- Create a relation schema **SM** with an attribute **m** that corresponds to the multivalued attribute **{m}** and an attribute, fk_S , corresponding to the primary key for entity **S**.
- The primary key for relation **SM** is the union of all its attributes.



- Add:**
1. a **foreign key constraint**: foreign key (fk_S) references **S**(k_S).
 2. a **referential integrity action**: on delete cascade.

 **If a relationship has a composite/multivalued attribute, “convert” the relationship to an entity before applying the reduction.**

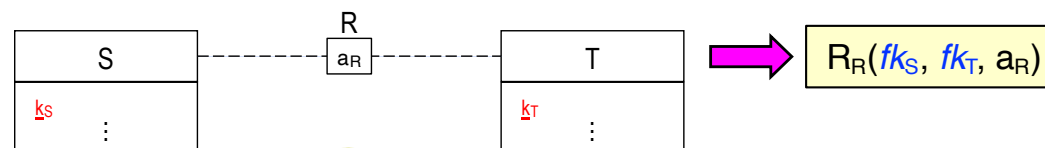
REDUCE RELATIONSHIPS

For each binary relationship **R**:

- Create a new relation schema R_R .
- Include as foreign key attributes in relation R_R the primary keys of the entities related by relationship **R**.
- Include attributes a_R of relationship **R**, if any, as attributes of relation R_R .
- The primary key for relation R_R is
 - *binary 1:1 relationship* $\Rightarrow$ the primary key for **either entity**.
 - *binary 1:N relationship* $\Rightarrow$ the primary key for the **entity on the N-side** of the relationship.
 - *binary N:M relationship* $\Rightarrow$ the **union of the primary keys** for the two participating entities.

Add: a **foreign key constraint** for each foreign key with the **referential integrity action on delete cascade**.

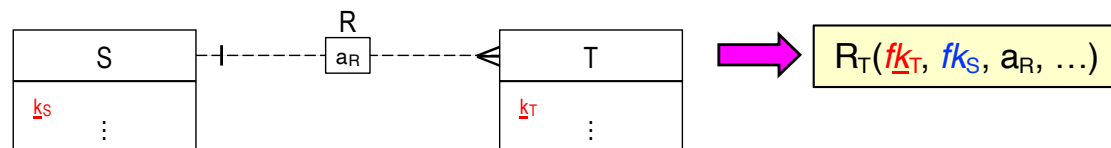
👉 **This reduction minimizes null values.**



RELATION SCHEMA COMBINATION

- For **1:1 relationships**, the relation schema for the relationship can be **combined with** the relation schema for **either entity**.
- For **1:N relationships**, the relation schema for the relationship can be **combined with** the relation schema for the **entity on the N-side**.

- Add:
1. a **foreign key constraint** for the foreign key.
 2. a **referential integrity action** for the foreign key that is **determined by the participation constraint of the entity into which the foreign key is placed**.
 - **partial**: on delete set null
 - **total**: on delete cascade



REDUCE EXCLUSION CONSTRAINTS

Option 1

Task(id, ..., *externalProjNo*, *internalProjNo*)

foreign key (*externalProjNo*) references ExternalProject(projectNo)

on delete cascade

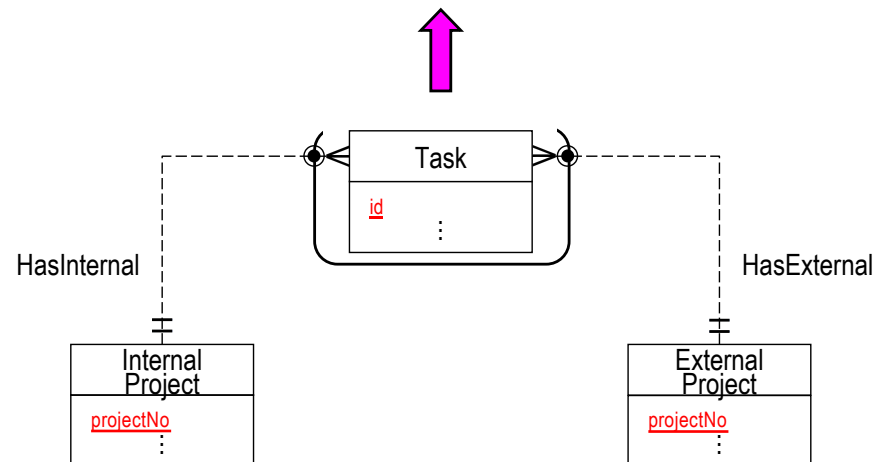
foreign key (*internalProjNo*) references InternalProject(projectNo)

on delete cascade

SCHEMA REDUCTION NOTE

An additional constraint is needed to ensure that only one of *externalProjNo* and *internalProjNo* has a value and the other is null.

check ((*externalProjNo* is not null and *internalProjNo* is null) or
(*internalProjNo* is not null and *externalProjNo* is null))



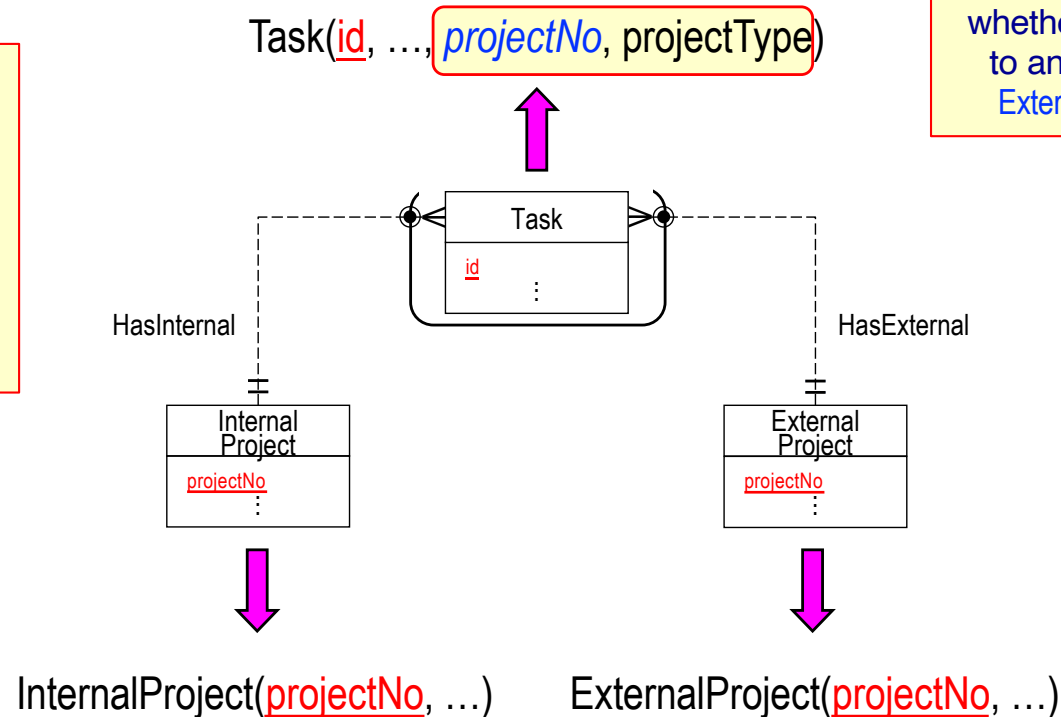
REDUCE EXCLUSION CONSTRAINTS (CONTD)

Option 2

SCHEMA REDUCTION NOTE

This reduction can only be used if the constrained entity types have keys that are of the same domain type.

Attribute `projectType` identifies whether the task is related to an `InternalProject` or an `ExternalProject` instance.



❗ Cannot specify referential integrity constraints or actions for **Task**!

BUT the exclusion constraint is **automatically enforced**.

(Can also use a combination of options 1 and 2.)